

Lección 9: Essential Metrics

Métricas Mínimas que Importan

Agenda

- Vanity Metrics vs Actionable Metrics (métricas vanidosas vs accionables)
 - Métricas Esenciales por Tier (nivel)
 - Implementación Práctica
 - Casos de Uso Reales
 - Anti-Patterns (anti-patronos)
-

El Problema

❌ Métricas Vanidosas:

 Lines of Code: +50,000 
 Commits per day: 47 
 Code coverage: 95% 
 Features shipped: 23 

🚨 PROBLEM: Todo está "verde" pero la app se cae en producción

✅ Métricas Útiles:

 Test Success Rate: 87%  (RED FLAG)
 Error Rate Production: 2.3%  (CRITICAL)
 Mean Time to Recovery: 45 min  (PROBLEM)

🎯 RESULT: Predicen problemas antes que lleguen a usuarios

Filosofía: Métricas que Guían Decisiones

Pregunta clave para cada métrica:

| ¿Esta métrica me ayuda a tomar una decisión específica?

❌ "95% coverage" → ¿Y qué hago con eso?

✅ "Tests críticos fallan 13%" → Refactoring urgente

❌ "Shipped 23 features" → ¿Son útiles?

✅ "Feature adoption: 3%" → Esta feature no sirve

❌ "47 commits/day" → ¿Son buenos?

✅ "Commits broke build: 23%" → Need quality gates

TIER 1: Métricas Críticas

Review Diario - Predicen problemas

1. Test Success Rate (Tasa de Éxito de Tests)

`tests_passing / total_tests = success_rate`

160 / 160 = 100% ✅

145 / 160 = 90.6% ⚠️

Acción:

< 95% → Parar features, arreglar tests

< 90% → Code freeze (congelamiento de código)

TIER 1: Build Success Rate

2. Build Success Rate (Tasa de Éxito de Build)

`successful_builds / total_builds = build_rate`

19 / 20 = 95% 

18 / 20 = 90% 

Acción:

< 95% → Investigar inmediatamente

< 90% → Auditoría de dependencias

TIER 1: Error Rate Production

3. Error Rate (Tasa de Errores en Producción)

`errors / total_requests = error_rate`

5 / 1000 = 0.5% 

23 / 1000 = 2.3% 

Acciones según tasa de error:

> 1% → Investigar inmediatamente

> 2% → Respuesta de emergencia

> 5% → Todos los recursos disponibles

Impacto directo en usuarios → Crítico para el negocio

TIER 2: Métricas Importantes

Review Semanal

4. Mean Time to Recovery (MTTR - Tiempo Medio de Recuperación)

`time_to_fix / number_of_issues = MTTR`

60 min / 4 issues = 15 min 

180 min / 4 issues = 45 min 

Mide: Velocidad de respuesta del equipo ante problemas

5. Deployment Frequency (Frecuencia de Despliegues)

deployments / time_period

1-2 deploys/día = Saludable ✅

2 deploys/semana = Velocidad baja ⚠️

TIER 3: Métricas de Contexto

Review Mensual

6. Technical Debt Ratio (Ratio de Deuda Técnica)

effort_to_fix_debt / total_effort

47 TODOs + 12 smells = Deuda alta ⚠️

5 TODOs + 2 smells = Deuda baja ✅

Acción:

> 30% → Programar sprint de deuda

> 50% → Refactorización arquitectónica

Implementación Práctica

Dashboard Simple (30 segundos):

```
echo "📊 Daily Metrics Check"
echo "Test Success: $(pnpm test --silent | grep pass)"
echo "Build Status: $(pnpm build && echo ✅ || echo ❌)"
echo "Lint Issues: $(pnpm lint --quiet | wc -l)"
```

package.json scripts:

```
{
  "scripts": {
    "metrics": "echo '📊 Health Check' &&
               npm run test:quick &&
```

```
        npm run build:check",
    "test:quick": "vitest --run --reporter=basic",
    "build:check": "tsc --noEmit"
  }
}
```

Caso de Uso 1: Startup Creciendo

Situación: 3 → 8 developers en 2 meses

Métricas que Importaron:

- ✓ Test Success: 98% → 87% → 95% (con intervención)
- ✓ Build Success: 95% → 78% → 92% (limpieza)
- ✗ Lines of Code: 15K → 45K (vanity, no action)
- ✗ Commits/day: 12 → 34 (vanity, no quality)

Resultado: Focus en métricas útiles previno crisis

Caso de Uso 2: Feature Launch

Situación: Shopping cart para Black Friday

Métricas que Salvaron el Día:

- ✓ Error Rate: 0.1% → 2.8% spike → ROLLBACK auto
- ✓ MTTR: 15 min average → Fast fixes
- ✗ Feature Adoption: 45% (medido después)

Resultado: Error rate detectó problema antes de quejas

Anti-Patterns

✗ **Métricas Complejas de Vanidad:**

```
# NO:  
"Average cyclomatic complexity: 4.7"  
# ¿Qué hago con eso?
```

```
# SÍ:  
"5 functions exceed threshold"  
# Actionable: revisar esas 5
```

❌ Dashboards Sobrecargados:

- 47 métricas diferentes → Analysis paralysis
- Nadie sabe qué es importante

SÍ: 3-5 métricas clave con thresholds claros

Herramientas Simples

Usar herramientas existentes:

```
vitest --run --reporter=verbose    # Tasa de éxito de tests  
tsc --noEmit                       # Éxito de build  
eslint . --format=compact          # Problemas de calidad
```

GitHub Actions:

```
- name: Check Red Flags  
  run: |  
    TEST_RATE=$(npm test | grep -o '[0-9]*%')  
    if [ "${TEST_RATE%?}" -lt 95 ]; then  
      echo "🚩 RED FLAG: Tests below 95%"  
      exit 1  
    fi
```

Ejercicio 1: Métrica Accionable - Cart Abandonment

Prompt:

Actúa como un desarrollador implementando métricas accionables para decisio

CONTEXT0: Actionable Metrics (métricas accionables) son métricas que guían decisiones específicas. Pregunta clave: "¿Esta métrica me ayuda a decidir q hacer?". Vanity Metrics (métricas vanidosas) como "47 commits/día" no dicta acción. Cart Abandonment Rate (tasa de abandono de carrito) mide cuántos usuarios agregan items pero NO compran: rate alto indica problema UX/precio Leading Indicators (indicadores adelantados) predicen problemas antes que afecten revenue. localStorage persiste conteos entre sesiones.

TAREA: Implementa tracking de Cart Abandonment Rate con thresholds accionab

MÉTRICA SPECIFICATION:

- Nombre: Cart Abandonment Rate
- Fórmula: $((addToCartCount - checkoutCount) / addToCartCount) * 100$
- Almacenamiento: localStorage para persistir entre sesiones
- Keys: 'addToCartCount', 'checkoutCount'

IMPLEMENTACIÓN:

- Función: trackAbandonmentRate()
- Ubicación: src/context/CartContext.tsx o src/features/shopping-cart/
- Leer counts desde localStorage (default 0 si no existe)
- Calcular abandonment rate con fórmula
- Log en console: "🔍 Cart Abandonment: X.X%"

THRESHOLDS ACCIONABLES:

- > 70% → console.warn('⚠️ ABANDONO ALTO: Revisar precios/UX')
- 50-70% → console.log('⚠️ Abandono moderado')
- < 50% → console.log('✅ Abandono normal')

LLAMAR FUNCIÓN:

- Al agregar item al carrito (incrementar addToCartCount)
- Al completar checkout (incrementar checkoutCount)

VALIDACIÓN: Agregar 5 items, completar 1 checkout → debe mostrar ~80% aband

Aprende: Thresholds accionables transforman números en

decisiones (> 70% = investigar UX)

Ejercicio 2: Dashboard Métrica - Conversion Rate

Prompt:

Actúa como un desarrollador creando dashboards simples con Binary Threshold

CONTEXT0: Binary Thresholds (umbrales binarios) hacen métricas accionables: claro cuándo está mal vs bien. Color-coding con emojis (🔴🟡🟢) comunica estado visualmente. Conversion Rate mide eficiencia de funnel: cuántos usuarios que agregan items completan compra. TIER 1 metrics se revisan diariamente. Console logging es dashboard simple para desarrollo `local`. Quick to Measure principle: dashboard debe tomar < 30 segundos revisar.

TAREA: Crea conversion rate dashboard simple en console con thresholds clar

MÉTRICA SPECIFICATION:

- Nombre: Conversion Rate (tasa de conversión)
- Fórmula: $(\text{checkouts} / \text{addToCart}) * 100$
- Almacenamiento: `localStorage ('addToCartCount', 'checkoutCount')`
- Output: console con formato estructurado

IMPLEMENTACIÓN:

- Función: `logConversionRate()`
- Ubicación: `src/features/shopping-cart/CartSummary.tsx` o similar
- Leer totals desde `localStorage`
- Calcular conversion rate
- Determinar status según thresholds

THRESHOLDS (Binary):

- $\geq 30\%$ → status = '🟢 EXCELENTE'
- $10-29\%$ → status = '🟡 NORMAL'
- $< 10\%$ → status = '🔴 CRÍTICO'

OUTPUT FORMAT (console.log):

 Conversion Rate: X.X% [STATUS]
Add to Cart: XX
Checkouts: XX

LLAMAR FUNCIÓN:

- Al montar `CartSummary` component (`useEffect`)
- 0 al completar checkout

VALIDACIÓN: Con 47 `addToCart` y 12 `checkouts` → debe mostrar "25.5% 🟡 NORMAL"

Aprende: Binary thresholds + visual indicators hacen

métricas inmediatamente accionables

Puntos Clave

1. **Actionable (accionables):** Métricas que dicen qué hacer
2. **Leading Indicators (indicadores adelantados):** Predicen problemas
3. **Binary Thresholds (umbrales binarios):** Claro cuándo está mal
4. **Quick to Measure (rápidas de medir):** < 30 segundos
5. **Focus (enfoque):** 3-5 métricas, no 47
6. **Daily Review (revisión diaria):** TIER 1 metrics
7. **Avoid Vanity (evitar vanidad):** LOC, commits count no ayudan